CARE-FLOW OPD

DATA MODEL & DATA DICTIONARY

Hospital Booking System for the Outpatient Department

Team Aura | Version 1.0 | 4 November 2026

Document purpose

This document defines the logical data model required to support the CareFlow OPD functional requirements. It identifies entities, attributes, primary keys, foreign keys, relationships, integrity rules, and requirement traceability. It is a logical design specification; it does not claim that a production database or backend has been implemented.

1. Design basis and modelling decisions

- **Actors:** Patient and Admin are system roles. Doctor is a profile/stakeholder entity and does not have a doctor-facing login in the current scope.
- **Unified account:** USER stores both Patient and Admin accounts; role distinguishes PATIENT from ADMIN. Only patient accounts use patient profile and booking functions.
- **Appointment vs token status:** APPOINTMENT.booking_status records the booking lifecycle (CONFIRMED/CANCELLED/COMPLETED). TOKEN.queue_status records the queue lifecycle (WAITING/CALLED/IN_OPERATION/COMPLETED/CANCELLED). This separation prevents one status field from representing two different concepts.
- **Availability:** A slot is bookable only when the doctor is active, the slot is open and in the future, and no active appointment occupies it. Availability is derived from slot and appointment data; it should not be maintained as a second conflicting status.
- **Visit reason privacy:** The optional visit reason is stored on APPOINTMENT, limited to 500 characters, and excluded from the general Admin dashboard. Access is restricted to the owning patient and explicitly authorized clinical personnel. The clinical role/workflow is not defined in the source and remains an implementation decision.
- **Follow-up:** A follow-up is a new APPOINTMENT linked to the completed appointment through previous_appointment_id. The original appointment and token history remain unchanged.
- **Account restriction:** USER.account_status stores ACTIVE or BANNED. Banning/restoring does not delete or modify existing appointments or tokens. ACCOUNT_ACCESS_AUDIT records who performed the action, reason, and timestamp.
- **Notifications:** NOTIFICATION belongs to one recipient user. Appointment-related notifications may reference an appointment. Read state is represented by read_at being NULL or populated.

2. Entity Relationship Diagram (ERD)

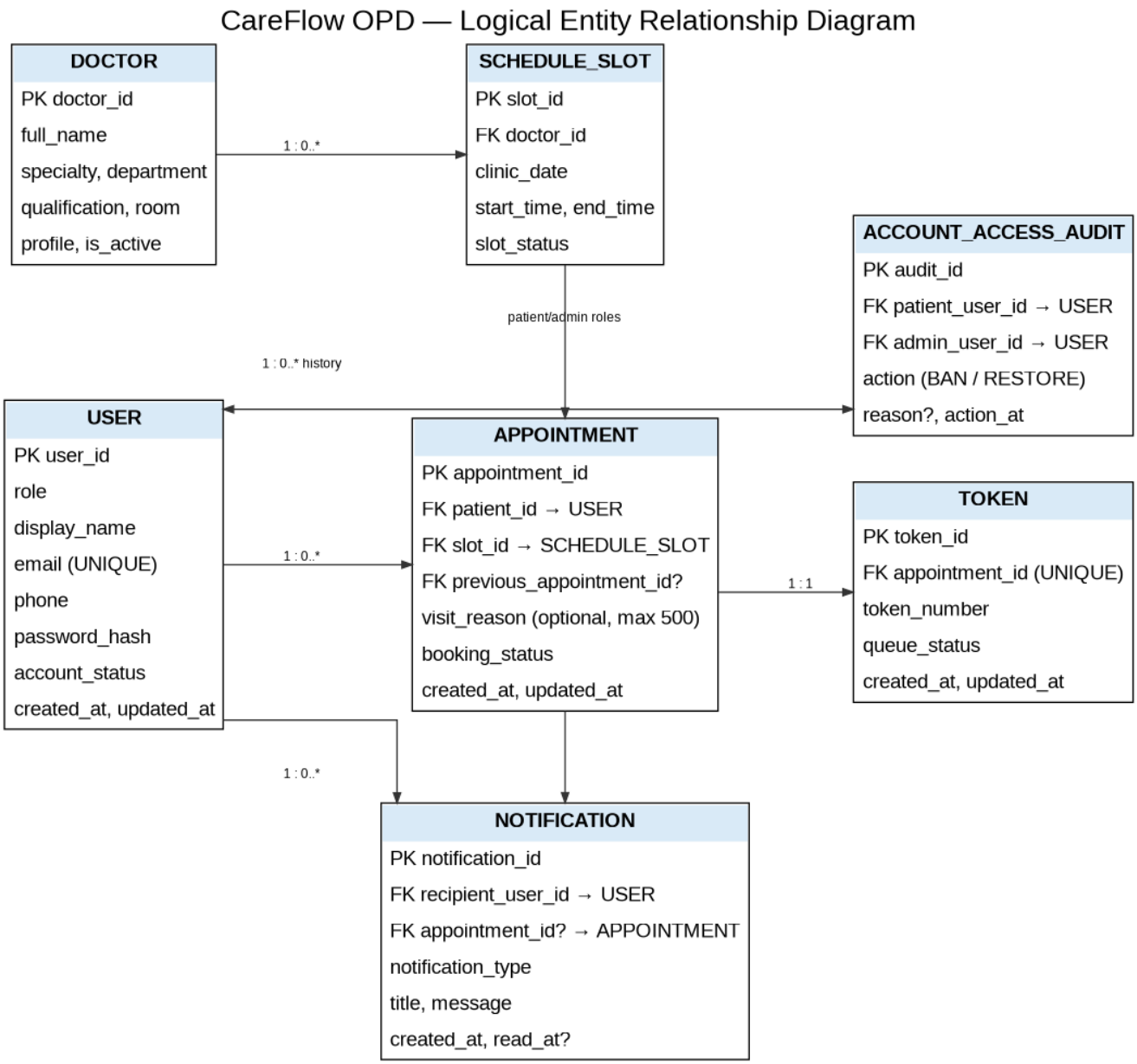


Figure 1. Logical ERD

3. Entity and relationship summary

Parent / source	Child / target	Cardinality	Relationship rule	FR
USER (Patient)	APPOINTMENT	1 to 0..*	One patient may create many appointments; each appointment belongs to exactly one patient.	FR-2, FR-3, FR-4, FR-8
DOCTOR	SCHEDULE_SLOT	1 to 0..*	One doctor may publish many schedule slots; each slot belongs to one doctor.	FR-1, FR-5
SCHEDULE_SLOT	APPOINTMENT	1 to 0..1 active booking	A slot may have no active appointment or one active appointment under the single-booking-per-slot rule. Historical cancelled appointments may remain linked.	FR-1, FR-2, FR-4, FR-5
APPOINTMENT	TOKEN	1 to 1	Each successfully confirmed appointment receives one token; each token belongs to one appointment.	FR-2, FR-3, FR-4, FR-6
APPOINTMENT (completed)	APPOINTMENT (follow-up)	1 to 0..*	A completed visit may be referenced by one or more new follow-up appointments; the original record is not overwritten.	FR-4
USER (recipient)	NOTIFICATION	1 to 0..*	A patient or Admin may receive many notifications; each notification has exactly one recipient.	FR-9
APPOINTMENT	NOTIFICATION	1 to 0..*	An appointment event may generate notifications. Some account notifications may have no appointment reference.	FR-9
USER (patient)	ACCOUNT_ACCESS_AUDIT	1 to 0..*	A patient account may have multiple ban/restore audit events.	FR-7
USER (admin)	ACCOUNT_ACCESS_AUDIT	1 to 0..*	An Admin may perform multiple account access actions; admin_user_id identifies the actor.	FR-7

4. Data dictionary

Data types are DBMS-neutral logical types. The team map them to the selected database types during implementation.

4.1 USER

Stores patient and Admin login identities, profile information, and account access state. Doctor is not a USER role in the current scope.

Attribute	Data type*	Key	Required	Description / constraint
user_id	UUID / CHAR(36)	PK	Yes	Unique account identifier.
role	VARCHAR(20)	—	Yes	PATIENT or ADMIN. Restrict allowed values with an enum/check constraint.
display_name	VARCHAR(120)	—	Yes	Name shown in the interface and appointment records.
email	VARCHAR(254)	UNIQUE	Yes	Login/registration email; must be unique.
phone	VARCHAR(30)	—	Confirm	Phone number shown in the patient profile; exact requiredness/format is not fully legible in supplied FR guide.
password_hash	VARCHAR(255)	—	Yes	Stores a password hash, never plaintext password. Hashing method belongs to security design.
account_status	VARCHAR(20)	—	Yes	ACTIVE or BANNED for patient access control.
created_at	DATETIME	—	Yes	Account creation timestamp.
updated_at	DATETIME	—	Yes	Last profile/account update timestamp.

- Business rule: a BANNED patient cannot book or reschedule. Existing appointments and tokens remain unchanged.
- Admin account provisioning method is not specified; do not assume public Admin registration.

4.2 DOCTOR

Stores doctor directory data and active/inactive state. Department is an attribute because the revised guide states department assignment is part of the doctor record and no separate department-management page exists.

Attribute	Data type*	Key	Required	Description / constraint
doctor_id	UUID / CHAR(36)	PK	Yes	Unique doctor identifier.
full_name	VARCHAR(120)	—	Yes	Doctor name.
specialty	VARCHAR(120)	—	Yes	Medical specialty.
department	VARCHAR(120)	—	Yes	Department assignment stored on the doctor record.
qualification	VARCHAR(255)	—	Confirm	Qualification information; exact validation rules are not supplied.
room	VARCHAR(60)	—	Optional	Consultation room, when available.
profile	TEXT	—	Optional	Doctor profile/biography shown in directory.
is_active	BOOLEAN	—	Yes	Inactive doctors are not available for new bookings.

- Do not hard-delete a doctor with appointment history; deactivate the profile and retain historical references.

4.3 SCHEDULE_SLOT

Represents one bookable doctor date/time interval. Each row is one slot, rather than storing a repeating weekly schedule as an opaque text field.

Attribute	Data type*	Key	Required	Description / constraint
slot_id	UUID / CHAR(36)	PK	Yes	Unique slot identifier.
doctor_id	UUID / CHAR(36)	FK → DOCTOR.doctor_id	Yes	Doctor who provides the slot.
clinic_date	DATE	—	Yes	Date of the clinic session.
start_time	TIME	—	Yes	Slot start time.
end_time	TIME	—	Yes	Slot end time; must be later than start_time.
slot_status	VARCHAR(20)	—	Yes	OPEN or CLOSED; patient-facing availability is also checked against active bookings.

- Unique/overlap rule: prevent duplicate or overlapping slots for the same doctor unless the hospital explicitly permits them.
- Schedule edits must not invalidate confirmed appointments silently. A conflict must be rejected or resolved through an approved process.

4.4 APPOINTMENT

Stores a patient booking, optional visit reason, cancellation/completion state, and follow-up linkage.

Attribute	Data type*	Key	Required	Description / constraint
appointment_id	UUID / CHAR(36)	PK	Yes	Unique appointment identifier.
patient_id	UUID / CHAR(36)	FK → USER.user_id	Yes	Must reference a USER whose role is PATIENT.
slot_id	UUID / CHAR(36)	FK → SCHEDULE_SLOT.slot_id	Yes	Selected doctor/date/time slot.
previous_appointment_id	UUID / CHAR(36)	Self-FK → APPOINTMENT.appointment_id	Optional	References the completed appointment when this record is a follow-up.
visit_reason	VARCHAR(500)	—	Optional	Patient-entered visit reason/medical-condition description; maximum 500 characters; restricted access.
booking_status	VARCHAR(20)	—	Yes	CONFIRMED, CANCELLED, or COMPLETED. Separate from token queue status.
created_at	DATETIME	—	Yes	Booking creation timestamp.
updated_at	DATETIME	—	Yes	Last booking update timestamp.

- A successful booking creates one appointment and one token in a single logical transaction.
- Prevent two active appointments from occupying the same slot. A cancelled appointment remains as history and no longer blocks the slot.
- For rescheduling, preserve the old booking as cancelled and create or update the booking. The new booking receives a token and an unavailable target slot must leave the old booking unchanged.
- For follow-up, create a new row and keep the original completed appointment unchanged.
- Do not expose visit_reason in the general Admin dashboard. The authorized clinical role/workflow must be specified before production.

4.5 TOKEN

Stores the queue identifier and backend-managed queue state associated with an appointment.

Attribute	Data type*	Key	Required	Description / constraint
token_id	UUID / CHAR(36)	PK	Yes	Unique token record identifier.
appointment_id	UUID / CHAR(36)	FK → APPOINTMENT.appointment_id; UNIQUE	Yes	Enforces one token per appointment.
token_number	VARCHAR(40)	UNIQUE within doctor/date queue	Yes	Display token assigned by backend using the approved numbering rule.
queue_status	VARCHAR(20)	—	Yes	WAITING, CALLED, IN_OPERATION, COMPLETED, or CANCELLED.
created_at	DATETIME	—	Yes	Token creation timestamp.
updated_at	DATETIME	—	Yes	Last token state update timestamp.

- Backend is the single source of truth for token status transitions; Patient and Admin views are read-only projections of that state.
- Admin must not manually create tokens, edit token status, call the next patient.

4.6 NOTIFICATION

Stores in-app notifications for Patient and Admin recipients, including unread/read state.

Attribute	Data type*	Key	Required	Description / constraint
notification_id	UUID / CHAR(36)	PK	Yes	Unique notification identifier.
recipient_user_id	UUID / CHAR(36)	FK → USER.user_id	Yes	The single user who receives the notification.
appointment_id	UUID / CHAR(36)	FK → APPOINTMENT.appointment_id	Optional	Related appointment, if the notification concerns an appointment event.
notification_type	VARCHAR(40)	—	Yes	Event category, e.g. registration, booking, cancellation, rescheduling, account access, or appointment status change.
title	VARCHAR(160)	—	Yes	Short notification heading.
message	VARCHAR(500)	—	Yes	Notification content; avoid including the private visit_reason.
created_at	DATETIME	—	Yes	Creation timestamp; used for newest-first ordering.
read_at	DATETIME	—	Optional	NULL means unread; timestamp means read.

- Unread count is calculated per recipient where read_at IS NULL.
- Mark-one-read sets read_at; mark-all-read sets read_at for all unread notifications belonging to the signed-in recipient.
- Create notifications only after the triggering event is committed

4.7 ACCOUNT_ACCESS_AUDIT

Records Admin actions to ban or restore patient account access, as required by FR-7.

Attribute	Data type*	Key	Required	Description / constraint
audit_id	UUID / CHAR(36)	PK	Yes	Unique audit event identifier.
patient_user_id	UUID / CHAR(36)	FK → USER.user_id	Yes	Patient account affected; referenced USER must have role PATIENT.
admin_user_id	UUID / CHAR(36)	FK → USER.user_id	Yes	Admin who performed the action; referenced USER must have role ADMIN.
action	VARCHAR(20)	—	Yes	BAN or RESTORE.
reason	VARCHAR(500)	—	Optional	Reason supplied for the ban; FR-7 makes this optional.
action_at	DATETIME	—	Yes	Timestamp of the action.

- Every ban and restore action must create an audit record.
- A ban/restore event changes USER.account_status only; it does not automatically change appointment or token status.

5. Integrity constraints and business rules

1. Primary keys uniquely identify every record. Foreign keys prevent orphaned appointments, tokens, notifications, and audit events.
2. USER.email is unique. A patient booking must reference an active PATIENT account.
3. DOCTOR.is_active must be true and SCHEDULE_SLOT.slot_status must be OPEN for a new booking.
4. A slot cannot be confirmed for a second active appointment. The check must be enforced by the backend/database, not only by hiding the slot in the interface.
5. Each successfully confirmed appointment has exactly one TOKEN. TOKEN.appointment_id is unique.
6. Appointment cancellation removes the booking from the active waiting queue and makes the slot available.
7. Rescheduling to an unavailable slot must fail without changing the existing confirmed appointment.
8. Follow-up creates a new appointment linked to the completed appointment; historical appointment and token data are retained.
9. Admin dashboard access to appointments is limited to identity/contact summary, doctor, appointment date/time, token number, and system-managed status. It excludes visit_reason.
10. Only the owning patient and explicitly authorized clinical personnel may access visit_reason.
11. Notifications are private to their recipient. The unread count is computed for the signed-in user only.
12. Ban/restore actions are auditable. Account restriction does not cascade-delete or mutate existing appointments/tokens.

6. Functional requirement traceability

Functional requirement	Supporting entities	Data support / expected behaviour
FR-1 View doctor information and availability	DOCTOR, SCHEDULE_SLOT	Active doctor filtering; display specialty, department, profile; show available slots only.
FR-2 Book appointment, optional reason, token	USER, DOCTOR, SCHEDULE_SLOT, APPOINTMENT, TOKEN	Validate active patient/doctor and slot; store optional reason ≤500 chars; create appointment + token; protect reason.
FR-3 View personal token and appointment status	USER, APPOINTMENT, TOKEN	Patient ownership filter; display own token and canonical queue status; do not expose another patient's token.
FR-4 Cancel/reschedule/follow-up	APPOINTMENT, SCHEDULE_SLOT, TOKEN	Cancellation and slot release; reschedule validation; self-referencing follow-up appointment; preserve history.
FR-5 Manage doctor information and schedules	DOCTOR, SCHEDULE_SLOT	Create/update/deactivate doctor; maintain slots; prevent schedule changes that conflict with confirmed bookings.
FR-6 Read-only Admin dashboard and records	USER, APPOINTMENT, TOKEN, DOCTOR, SCHEDULE_SLOT	Aggregate counts and search/filter; token/status fields read-only; no manual queue actions.
FR-7 Ban/restore patient account	USER, ACCOUNT_ACCESS_AUDIT	Change ACTIVE/BANNED; store actor/reason/time; preserve existing appointment/token records.
FR-8 Register/sign in/manage profile	USER	Unique email, authentication data, display name and phone profile fields; banned access restriction.
FR-9 In-app notifications	NOTIFICATION, USER, APPOINTMENT	Recipient-specific bell/unread count/list; newest-first; mark read; generate event notifications after commit.

7. Implementation boundary

This is a logical data model for the target system. The current project is a frontend-focused academic deliverable. The document does not claim a deployed database, backend authorization, transactional booking lock, persistent notification service, or clinical access-control workflow. These controls must be implemented and tested in the backend before the system is used with real patients.